Acadlore Transactions on AI and Machine Learning
https://www.acadlore.com/journals/ATAIML

Observability and Causal Tracing for Agentic Data Pipelines: Span Semantics, Replay, and Hops-To-Root-Cause

Muralikrishna Dudaka*

Independent Researcher, United States

* Correspondence: Muralikrishna Dudaka (reachmuralid@gmail.com)

Received: 06-30-2026 **Revised:** 07-31-2026 **Accepted:** 08-10-2026

Citation: M. Dudaka, "Observability and causal tracing for agentic data pipelines: Span semantics, replay, and hops-to-root-cause," *Acadlore Trans. Mach. Learn.*, vol. 5, no. 3, pp. 262–269, 2026. https://doi.org/10.56578/ataiml050305.

Abstract: Agentic data pipelines, in which large language models select and invoke tools through the Model Context Protocol, consume tool outputs, and iteratively execute multi-step analytical or operational workflows, are increasingly being deployed in production environments. However, the observability infrastructure required to diagnose failures in such systems remains underdeveloped. Conventional distributed tracing effectively captures service-to-service execution but often represents large language model invocations as opaque spans and fails to preserve causal relationships across large language model-tool boundaries. Consequently, incident diagnosis can require an agent's execution trajectory to be reconstructed manually from chronologically ordered spans. To address this limitation, causal span linking across large language model and tool invocations was defined as a first-class observability primitive for agentic data pipelines. Hops-to-root-cause was introduced as the directed acyclic graph distance between a symptom span—the earliest span tagged error=true—and the identified causal span, with deterministic tie-breaking applied. The proposed approach was evaluated on a synthetic corpus comprising 20 incidents generated using a fully disclosed construction protocol. Compared with a flat-span baseline, causally linked tracing reduced the mean hops-to-root-cause from 6.4 to 1.5, corresponding to a reduction by a factor of 4.3. The greatest improvements were observed for incidents involving multi-hop tool chains. Causal span linking was further complemented by deterministic replay and by a human-artificial intelligence collaborative diagnostic workflow. Together, causal span linking, deterministic replay, and human-artificial intelligence collaborative diagnosis were established as complementary observability primitives for improving the reproducibility, interpretability, and efficiency of root-cause analysis in agentic data pipelines.

Keywords: Observability; Distributed tracing; Agentic systems; Large language models; Model context protocol; Root-cause analysis; Opentelemetry; Hops-to-root-cause

1 Introduction

Agentic data pipelines interleave heterogeneous compute according to a fundamentally different configuration from traditional service-to-service architectures. A large language model call may take hundreds of milliseconds and produce a structured tool invocation through the Model Context Protocol [1]; the tool call returns a result; the large language model reasons over the result and may invoke another tool; the cycle continues until the agent produces a terminal answer. This pipeline shape—chains of large language model-tool pairs—imposes observability requirements that traditional tracing infrastructure, built around synchronous service-to-service request propagation, does not meet.

The distributed tracing literature traces its lineage to Dapper [2] and the X-Trace framework [3], with OpenTelemetry [4] consolidating the canonical model for the contemporary ecosystem. Pivot Tracing [5] established causal observation across service boundaries as a first-class tracing primitive; the same principle applies, with new urgency, at the large language model-tool boundary in agentic pipelines. A typical instrumented agentic pipeline produces a flat span list in which the large language model call is a single leaf span and each tool call is a sibling, with no causal link expressing that a particular tool call was caused by a specific large language model completion. Practitioner observability platforms, including LangSmith [6], LangFuse [7], and Phoenix [8], document that root-cause analysis on agentic incidents commonly takes hours, with the bottleneck being trajectory reconstruction rather

than any individual span's lookup. While these platforms capture span data and provide basic lineage views, they share a common structural limitation: tool-call spans are recorded as siblings under an agent-loop parent rather than as causal children of the large language model completion that produced them. This means that trace visualizations present co-occurrence without causality, leaving the operator unable to answer the question "which large language model completion triggered this tool failure?" without manually reading every completion in sequence.

Three contributions are advanced. First, causal span linking across large language model and tool boundaries is defined: each tool-call span declares its parent as the large language model completion span whose reasoning produced the call. Second, hops-to-root-cause is defined as the directed acyclic graph distance from the symptom span to the causal span, with deterministic tie-breaking. Third, a 20-incident synthetic corpus with a fully disclosed construction protocol validates the metric, showing a 4.3× reduction in mean hops under causal tracing.

2 Background

2.1 Distributed Tracing Primitives

The canonical tracing model introduced by Dapper [2] and X-Trace [3] establishes the span as the fundamental observability unit. Each span carries a unique trace identifier, a parent span identifier, start and end timestamps, a status, and a key-value attribute set. OpenTelemetry [4] consolidates this model across languages and vendors, with semantic conventions for generative artificial intelligence now evolving to standardize large language model-call attributes. Production tracing systems ingest OpenTelemetry-format traces; the underlying span model is uniform. The model handles fan-out at service boundaries well; it does not natively handle causality mediated by an opaque reasoning component—precisely the pattern that agentic pipelines introduce.

2.2 Agentic Pipeline Anatomy

An agentic data pipeline begins with a user prompt or external trigger. The agent constructs an initial prompt and issues a large language model call using the ReAct pattern [9]; the large language model returns a completion that may include a tool-call directive via the Model Context Protocol [1]; the agent invokes the tool and incorporates the result into the next large language model call. The pipeline is a sequence of large language model-tool pairs terminated by a final large language model call. Several structural features distinguish this pipeline from traditional distributed architectures: call depth is bounded by the agent's maximum-turn limit (commonly 10 to 50); branching is rare (the call tree is mostly a linear chain); non-determinism is endemic, complicating replay [10, 11]. The broader agent survey landscape [10, 12, 13] confirms that multi-step tool-using agents are now common in production, making observability infrastructure for these pipelines a pressing practical need.

3 Span Semantics for Large Language Model and Tool Calls

3.1 Large Language Model Calls as First-Class Spans

The large language model call is treated as a first-class span with a rich attribute set: model identifier, prompt content or hash, completion content or hash, input and output token counts, latency, finish reason, and sampling parameters. The span's attributes are sufficient to reconstruct, with the large language model provider, the exact call that was made. The OpenTelemetry semantic conventions for generative artificial intelligence [4] are evolving to standardize these attributes; the practitioner toolkit converges on a similar set across LangSmith [6], LangFuse [7], and Phoenix [8]. Chain-of-thought research [11] establishes that the reasoning content of large language model completions is the causal antecedent of subsequent tool calls—precisely the link that causal span instrumentation makes explicit. Table 1 provides a structured comparison of causal-link tracing against these platforms across four dimensions: trace structure (flat sibling list vs. causal directed acyclic graph), causal relationship modeling (none vs. explicit parent-child edges at the large language model-tool boundary), root-cause analysis capability (manual hop-by-hop vs. directed acyclic graph traversal), and replay support (absent vs. deterministic from captured attributes). The proposed framework is the only approach that provides all four capabilities in combination, addressing the specific gap identified in the introduction.

3.2 Tool Calls as Causal Children

The central instrumentation design choice is to instrument each tool-call span as a causal child of the large language model completion span that produced it, rather than as a sibling under a common agent-loop parent. The agent loop must attach the context of the active large language model completion span as the parent for the tool call's instrumentation when dispatching the call through the Model Context Protocol client transport layer [1]. In Model Context Protocol-based deployments, both attachment points live in the client transport layer where tool dispatch already occurs. The Pivot Tracing discipline [5] advocates the same approach at the inter-service level; the same logic applies at the large language model-tool boundary. Multi-agent collaboration frameworks [14] and agentic tuning research [15] both confirm that causal linkage is necessary for meaningful trajectory analysis in complex agent systems.

Table 1. Comparison of causal-link tracing with existing observability platforms

Dimension	LangSmith	LangFuse	Phoenix	Proposed Framework
Trace structure	Flat sibling list	Flat sibling list	Flat sibling list	Causal directed acyclic graph
Causal relationship modeling	None	None	None	Explicit parent-child edges at the large language model-tool boundary
Root-cause analysis	Manual hop-by-hop	Manual hop-by-hop	Manual hop-by-hop	Directed acyclic graph traversal
Replay support	Absent	Absent	Absent	Deterministic from captured attributes

4 Hops-To-Root-Cause

4.1 Definition and Properties

Hops-to-root-cause is defined precisely. Given a trace directed acyclic graph T with span set S and parent map $P : S \to S \cup \{\emptyset\}$, the symptom span S_{sym} is the chronologically first span in S tagged error=true, with ties broken by lexicographic span identifier. The causal span S_{cause} is the deepest ancestor of S_{sym} whose internal state caused the failure observed at S_{sym}. This definition presupposes construction-time ground-truth annotation of the causal span. Such annotation is available by construction in the synthetic corpus. Production use requires operator-provided annotation at trace-construction time; the metric is not self-computing from unlabeled production traces. The hops-to-root-cause metric is the directed acyclic graph distance $d(S_{\text{sym}}, S_{\text{cause}})$, measured as the number of parent traversals required to reach S_{cause} from S_{sym}. The metric is deterministic under the construction and workload-agnostic—it depends only on the trace structure and error annotations. Root cause identification research [16] establishes that causal-graph traversal substantially outperforms flat correlation on the same incident data.

Three practical pathways for causal annotation in production are proposed. First, automated detection: spans tagged error=true can be propagated upward through the directed acyclic graph using a lightweight static analysis pass; if a tool-call span is the chronologically first error, its parent large language model completion span is a candidate causal span. This heuristic is correct for the majority of single-fault incidents. Second, operator feedback: observability dashboards can surface the candidate causal span and prompt the on-call operator to confirm or redirect; this labeled signal can be stored in a causal annotation store keyed by trace identifier. Third, incident report integration: post-incident review documents typically identify root causes at the component level; a structured incident report template that includes a "causal span identifier" field bridges the gap between incident management and trace storage. Together, these pathways transform causal annotation from a purely manual task into a semi-automated workflow that scales with operational volume.

4.2 Flat-Span Baseline and the Combinatorial Problem

The natural baseline is the flat-span analysis pattern, in which the operator walks the chronological span list backward from the symptom span. The hops-to-root-cause for the flat-span baseline is the position of the causal span in the chronological list counted backward from the symptom. For a trace at directed acyclic graph depth d with branching factor b, the flat-span position is approximately $k \approx \Theta(d \times b)$. The search problem is that the operator cannot distinguish causal relationships from temporal coincidence without reading the large language model's completion text at each step. The Reflexion framework [17] highlights how this ambiguity compounds in self-correction loops, which are common in multi-step analytical agents.

4.3 Limitations

The hops-to-root-cause metric as defined does not account for retry spans: if an agent retries a failed tool call, the retry span may be the chronologically first error-tagged span while the original invocation is the true causal antecedent. A retry-aware variant would suppress spans tagged as retries when selecting ssym. This extension is left for future work. More broadly, the current evaluation is limited to a synthetic corpus, which cannot capture the full diversity of production failure modes. This limitation is explicitly acknowledged as the primary limitation of the present work. The corpus was constructed with five incident classes to demonstrate that the metric behaves consistently across qualitatively distinct failure types, but class diversity in production environments is substantially higher. The evaluation also does not address scenarios where the causal span is ambiguous or where multiple concurrent agent turns contribute to a compound failure.

5 A Synthetic 20-Incident Corpus

5.1 Construction Protocol

A synthetic 20-incident corpus was constructed with five incident classes and four incidents per class, with pipeline depths drawn uniformly from {2, 3, 5, 8} large language model-tool turns. The corpus comprises five incident classes: timeout cascade (downstream tool exceeds time budget), schema drift (tool returns unexpected schema; large language model hallucinates column name), tool-argument malformation (large language model produces malformed arguments; tool returns structured error), downstream rate limit (high-frequency capability exceeds server-side rate limit), and large language model hallucination of nonexistent column. Faults were injected via Python decorators on synthetic tools; the ground-truth causal span was known by construction. Traces were emitted in both flat-span and causally linked form. The corpus was made reproducible from a fixed random seed. All results are outputs of the analytical model and simulator developed for this study; they are not measurements of any production system.

The corpus is sized for an existence proof of the metric's reduction factor, not for statistical power; class sizes are uniform to prevent class imbalance from confounding the comparison. Specifically, the four pipeline depths {2, 3, 5, 8} were chosen to span the range from minimal (two large language model-tool turns, typical of simple lookup agents) to deep (eight turns, representative of analytical agents that iteratively refine queries). Each depth was paired with each incident class to ensure that the reduction factor is not an artifact of a single depth regime. The incident classes were selected to cover the three most common failure modes reported by practitioners using LangSmith and LangFuse: schema mismatches, timeout propagation, and large language model-introduced argument errors. A corpus of 20 incidents provides limited statistical power and is insufficient to support broad generalization; the validation is intended as a proof of metric existence, and a production-scale evaluation requires operator-labeled traces from live deployments. Future work will address this by partnering with enterprise deployments to collect real incident traces and evaluate the metric on datasets with hundreds of incidents across a wider class distribution. Additionally, experiments examining variations in the branching factor (fan-out > 1) and retry behavior are identified as important extensions.

5.2 Results

Hops-to-root-cause is computed for both trace forms. Table 2 reports the mean and 90th-percentile hops-to-root-cause per incident class. Mean hops-to-root-cause falls from 6.4 under flat-span analysis to 1.5 under causal-link traversal, a factor of 4.3 across the 20-incident corpus. The largest reductions occur in the rate-limit (4.7×) and large language model hallucination (4.5×) classes. All incident classes share the same four pipeline depths ({2, 3, 5, 8}) by design; the larger reductions in these two classes reflect the fact that their fault patterns tend to manifest later in the execution trace, increasing the flat-span hop count while the causal-link distance remains bounded by directed acyclic graph depth. The smallest reduction is in tool-argument malformation (3.1×) because that class has shallow trajectories. Recall is exact in both forms—the ground-truth causal span is found in every incident. The metric measures investigation efficiency, not correctness. Table 3 reports the causal-link instrumentation impact by pipeline depth.

Table 2. Mean and 90th-percentile hops-to-root-cause by incident class

Incident Class	Flat Mean	Flat P90	Causal Mean	Causal P90	Reduction (Mean)
Timeout cascade	6.0	11	1.5	3	4.0×
Schema drift	7.5	13	1.8	3	4.2×
Tool-argument malformation	2.5	5	0.8	2	3.1×
Downstream rate limit	7.0	13	1.5	3	4.7×
Large language model hallucination	9.0	14	2.0	4	4.5×
Aggregate (20 incidents)	6.4	14	1.5	3	4.3×

Note: Data are illustrative, based on a 20-incident corpus and analytical model data.

Table 3. Causal-link instrumentation impact by pipeline depth

Pipeline Depth (Large Language Model-Tool Turns)	Flat Mean Hops	Causal Mean Hops	Reduction (Mean)	Directed Acyclic Graph Traversal Advantage
2	2.8	0.9	3.1×	Minimal—shallow trajectory
3	4.5	1.2	3.8×	Moderate
5	7.2	1.5	4.8×	Substantial
8	10.8	1.8	6.0×	Largest—deep multi-hop chain

Note: Data are illustrative analytical model data.

6 Replay and Operator Workflow

Causal span linking supports a second observability primitive: deterministic replay. Given a captured trace with full large language model-call attributes and tool-call typed arguments and results, a replay engine reconstructs the agent's trajectory step by step and re-executes it against a non-production environment. The Toolformer study [18] and the ToolLLM benchmark [19] both highlight the importance of traceable tool invocations for agent debugging; replay provides the mechanism to verify remediation without exposing the production environment to a modified agent. The Voyager and Reflexion frameworks [17, 20] demonstrate that agent self-improvement loops require exactly this kind of replay capability to be operationally safe.

For replay to be deterministic—meaning exact trajectory reconstruction by replaying captured completions and tool outputs, rather than re-invoking the model (which may remain nondeterministic)—the following information must be stored at trace time: (i) the full large language model call record, including model identifier and version, sampling parameters (temperature, top-p, and max tokens), and the complete prompt and completion text, or a retrievable content-addressed object thereof (a hash alone is sufficient for content verification but cannot reconstruct the text; deterministic replay requires the full content or a content-addressed store from which it can be retrieved); (ii) all tool-call arguments and return values in their typed, serialized form; (iii) the wall-clock timestamp and monotonic timestamp of each span to support time-conditioned tool behavior. Model version changes are handled by recording the model identifier as a span attribute; a replay engine that detects a version mismatch can flag the replay as approximate and surface the discrepancy to the operator rather than silently producing incorrect results. Tool execution environment consistency is maintained by replaying against a snapshot of the external tool state taken at the time of the original trace; in practice, this means either mocking tool responses from the stored return values (the most common approach) or maintaining a versioned test environment that mirrors the production tool state at a specific timestamp.

As a concrete example, consider a timeout cascade incident: the replay engine loads the stored tool return values for each turn, re-invokes the agent with the original prompt and model version, and verifies that the same timeout is reproduced; the operator then modifies the timeout threshold in the agent configuration and re-runs the replay to confirm the fix. The human-artificial intelligence collaboration framing centers on the operator workflow enabled by these primitives. Faced with a production incident, the operator examines the visualization of the trace directed acyclic graph, follows parent pointers from the symptom span to the causal span, reads the causal span's attributes, and decides on a remediation. The replay primitive verifies the remediation by re-running the captured trace against the modified agent. The two primitives together transform the agent's behavior from an opaque process into an investigable one, placing the operator in the position of steering the agent's evolution rather than merely observing its failures. In enterprise deployments where agentic interfaces serve security-sensitive analytical workloads, this accountability infrastructure is operationally necessary.

7 Validation via Trace Simulator

A Python trace simulator was constructed to generate the 20-incident corpus and compute the hops-to-root-cause metric for both trace forms. The simulator generates each incident by constructing a synthetic agent loop with the specified depth, injecting a fault at the specified causal span, and emitting the trace in both flat-sibling and causally linked form. The hops-to-root-cause computation traverses the trace in both forms and records the distance to the ground-truth causal span. The simulator is approximately 600 lines of Python, runs in under one minute, and is reproducible from a fixed random seed. Operators interested in evaluating the discipline on their own workloads can adapt the simulator by replacing the synthetic incident classes with real-incident replays extracted from production trace storage.

8 Broader Implications for Agentic Infrastructure

As agentic pipelines are deployed in regulated environments—financial analytics, healthcare data platforms, and compliance-critical operational workflows—the question of accountability becomes central. The augmented language models survey [21] identifies observability as a critical gap in the deployment of tool-using agents at scale; this research provides a concrete framework for closing that gap. The study on adaptive query execution [22] confirms that lakehouse-native analytical pipelines, which are a common deployment context for agentic data pipelines, benefit directly from the causal-link instrumentation proposed in this study.

To ground the proposed framework in concrete applied artificial intelligence scenarios, three representative use cases are described. In enterprise analytics agents, a multi-step agent that queries a lakehouse, applies transformations, and generates a business intelligence report may fail due to schema drift in an upstream table; causal span linking allows the analytics platform operator to identify the failing schema-fetch span directly rather than tracing through dozens of transformation steps. In automated data processing systems, pipelines that ingest, validate, transform, and route data using large language model-guided decision logic benefit from causal tracing when downstream validation failures must be traced to the upstream large language model routing decision that sent malformed records to the wrong schema validator. In decision-support applications, where a large language model agent synthesizes evidence from multiple retrieval tool calls and produces a recommendation, causal tracing enables auditors to verify exactly which retrieved evidence influenced the recommendation, supporting regulatory compliance requirements in healthcare and financial services contexts.

Multi-agent deployments—where one agent invokes another as a tool through the Model Context Protocol [1]—generalize naturally to the causal-link discipline. The sub-agent's trigger span declares its parent as the invoking agent's large language model completion span that produced the invocation, and the trace directed acyclic graph spans both agents. The AgentVerse [14] and AgentTuning [15] frameworks both operate in this multi-agent regime; causal-link instrumentation is the prerequisite that makes their execution trajectories investigable. The OpenTelemetry generative artificial intelligence semantic conventions [4] are the natural standardization target for the span attribute schema proposed in this study. Table 4 presents the span attribute schema for large language model calls and tool calls in causally linked tracing.

Table 4. Span attribute schema for large language model calls and tool calls in causally linked tracing

Span Type	Attribute Name	Value/Type	Purpose
Large language model call	gen_ai.model	string	Identifies model version for replay
Large language model call	gen_ai.prompt_hash	Secure Hash Algorithm 256 (SHA-256)	Enables content verification without exposure
Large language model call	gen_ai.completion_hash	SHA-256 string	Identifies completion for deterministic replay
Large language model call	gen_ai.input_tokens	integer	Cost tracking and anomaly detection
Large language model call	gen_ai.output_tokens	integer	Cost tracking and reasoning-depth indicator
Large language model call	gen_ai.finish_reason	enum (stop/tool_call/length)	Indicates terminal vs. tool-calling completion
Tool call	mcp.tool_name	string	Capability identifier for audit log
Tool call	mcp.tool_arguments_hash	SHA-256 string	Typed argument fingerprint for replay
Tool call	mcp.result_summary	structured JavaScript Object Notation (JSON)	Schema shape for filtering without raw data
Tool call	parent_span_id	large language model completion span identifier	Causal link—the core instrumentation change
Tool call	error	boolean	Symptom-span detection for hops metric

9 Conclusion

Agentic data pipelines demand an observability discipline that existing distributed tracing infrastructure does not natively provide. Causal span linking across large language model and tool boundaries—instrumenting each tool-call span as a child of the large language model completion span that caused it—is a small implementation change with structurally large operational consequences. It transforms the trace from a flat list of co-occurring spans into a directed acyclic graph that expresses the agent's causal trajectory. In the synthesized 20-incident corpus with complete disclosure of the construction process, causal tracing decreases average hops-to-root-cause from 6.4 to 1.5—a ratio of 4.3—relative to flat-span investigation. The deterministic replay primitive verifies remediations against captured production traffic. Together, these primitives put the human operator in the position of steering the autonomous agent's evolution with evidence, accountability, and control.

Data Availability

The results supporting the research findings are derived from the analytical model and trace simulator described in Section 5 and Section 7. The simulator and synthetic corpus are not publicly archived; the construction protocol and fixed random seed documented in Section 5 allow independent reproduction of the corpus and results.

Acknowledgements

The author thanks the open-source observability community for their work on OpenTelemetry and the practitioner documentation that informed this study.

Conflicts of Interest

The author declares no conflicts of interest.

Declaration on the Use of Generative AI and AI-assisted Technologies

The author declares that generative AI tools were used in the preparation of this manuscript for language editing and proofreading purposes. The author remains fully responsible for the content of the work and for ensuring its accuracy, originality, and compliance with ethical and publishing standards.

References

[1] Model Context Protocol, "Model context protocol specification," 2024. https://modelcontextprotocol.io/specification/2025-11-25

[2] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure," Google, Tech. Rep. dapper-2010-1, 2010. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/

[3] R. Fonseca, G. Porter, R. H. Katz, and S. Shenker, "X-Trace: A pervasive network tracing framework," in *4th USENIX Symposium on Networked Systems Design & Implementation (NSDI 2007)*, Cambridge, MA, 2007. https://www.usenix.org/legacy/event/nsdi07/tech/full_papers/fonseca/fonseca.pdf

[4] OpenTelemetry, "OpenTelemetry semantic conventions for generative AI," 2024. https://opentelemetry.io/docs/specs/semconv/gen-ai/

[5] J. Mace, R. Roelke, and R. Fonseca, "Pivot tracing: Dynamic causal monitoring for distributed systems," in *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015, pp. 378–393. https://doi.org/10.1145/2815400.2815415

[6] LangChain, "LangSmith documentation: Tracing and observability for LLM applications," 2024. https://docs.smith.langchain.com/

[7] LangFuse, "Langfuse documentation: Open source LLM engineering platform," 2024. https://langfuse.com/docs

[8] Arize AI, "Phoenix: Observability for LLM applications," 2024. https://docs.arize.com/phoenix

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," *arXiv Preprint*, p. arXiv:2210.03629, 2023. https://doi.org/10.48550/arXiv.2210.03629

[10] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, no. 2, pp. 1–44, 2025. https://doi.org/10.1007/s11432-024-4222-0

[11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022, pp. 24 824–24 837. https://doi.org/10.52202/068431-1800

[12] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems*, 2024, pp. 126 544–126 565. https://doi.org/10.52202/079017-4020

[13] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, and J. Tang, "Agentbench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 52 989–53 046. https://arxiv.org/abs/2308.03688

[14] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C. M. Chan, and J. Zhou, "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 20 094–20 136.

[15] A. Zeng, M. Liu, R. Lu, B. Wang, X. Liu, Y. Dong, and J. Tang, "AgentTuning: Enabling generalized agent abilities for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3053–3077. https://doi.org/10.18653/v1/2024.findings-acl.181

[16] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "Cloudranger: Root cause identification for cloud native systems," in *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, Washington, DC, USA, 2018. https://doi.org/10.1109/CCGRID.2018.00076

[17] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, 2023, pp. 8634–8652. https://doi.org/10.52202/075280-0377

[18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Adv. NeurIPS*, vol. 36, pp. 68 539–68 551, 2023.

[19] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 9695–9717.

[20] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv Preprint*, p. arXiv:2305.16291, 2023. https://doi.org/10.48550/arXiv.2305.16291

[21] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, J. Dwivedi-Yu, A. Celikyilmaz *et al.*, "Augmented language models: A survey," *arXiv Preprint*, p. arXiv:2302.07842, 2023. https://doi.org/10.48550/arXiv.2302.07842

[22] M. Xue, Y. Bu, A. Somani, W. Fan, Z. Liu, S. Chen, H. van Hovell, B. Samwel, M. Mokhtar, R. Korlapati *et al.*, "Adaptive and robust query execution for lakehouses at scale," in *Proceedings of the VLDB Endowment*, vol. 17, no. 12, 2024, pp. 3947–3959. https://doi.org/10.14778/3685800.3685818